

Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológica

PARADIGMAS DE PROGRAMACIÓN



TRABAJO PRÁCTICO N°2

TEMA: Magos vs Mortífagos

GRUPO DELTA

INTEGRANTES:

- Albornoz Diego Fernando
- Cancian Facundo Leonel
- Esmoris Matias Ariel
- Gutierrez Edgardo Damian
- Vieiro Corallo Facundo Ivan
- Sierra Jazmín Victoria

1. Introducción

El presente trabajo práctico tiene como objetivo principal diseñar e implementar un simulador de batallas por turnos basado en la temática “Batalla de Magos contra Mortífagos”.

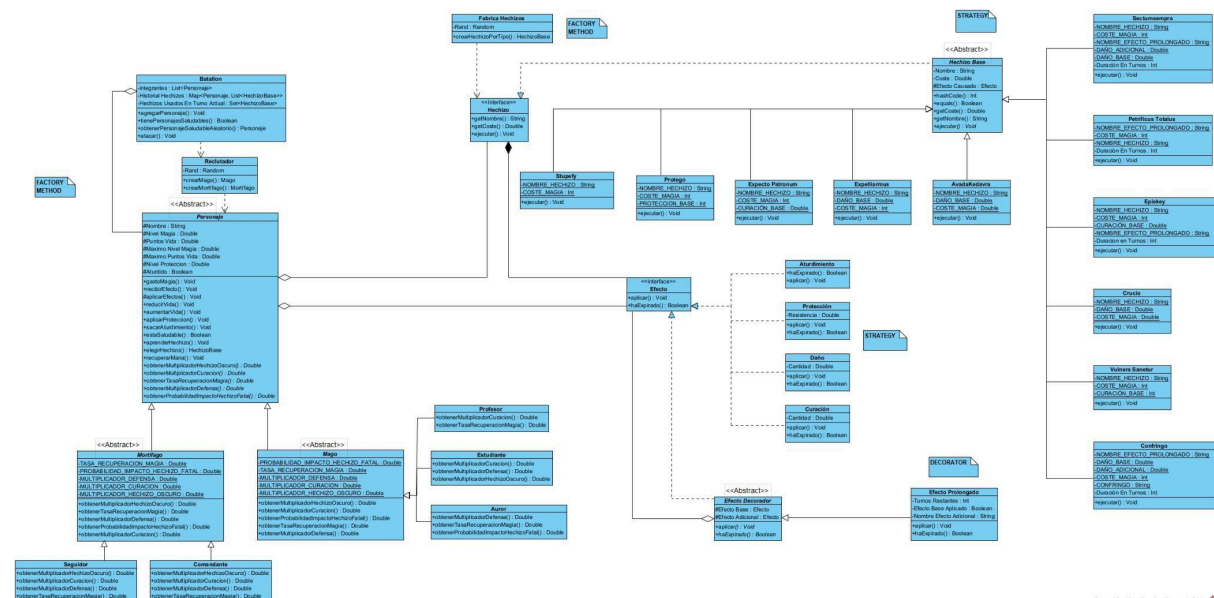
El desarrollo de este simulador busca poner en práctica y consolidar los conceptos fundamentales del Paradigma Orientado a Objetos (POO). A lo largo del proyecto, se modelaron las distintas entidades del dominio utilizando herencia para definir la jerarquía y las características específicas de los combatientes, y polimorfismo para resolver dinámicamente el comportamiento de los hechizos (ataque, defensa y curación) y sus respectivos multiplicadores, evitando la dependencia de estructuras condicionales rígidas.

Adicionalmente, el sistema integra el uso de diversas colecciones (List, Set y Map) para la organización eficiente de los batallones y el cumplimiento estricto de las reglas de combate. Finalmente el proyecto fue validado mediante pruebas unitarias, garantizando que las mecánicas del juego funcionen según las especificaciones.

2. Desarrollo y Decisiones de Arquitectura

2.1.Diagrama de clases

A la hora de diseñar el Diagrama de Clases, para organizarnos mejor, lo pensamos en torno a 3 ejes principales: Personaje, Hechizo y Efecto. El resto de clases son derivados de estos tres.



2.2.1. Creación y Jerarquía de Personajes

Los combatientes fueron modelados a través de una clase abstracta principal denominada “Personaje”, de la cual se desprenden dos ramas abstractas: “Mago” y “Mortifago”. A su vez, estas derivan en subclases concretas, siendo “Estudiante”, “Profesor”, “Auror” para los Magos, y “Seguidor” y “Comandante” para los Mortifagos.

Para aislar la lógica de creación, se utilizó la clase `Reclutador` aplicando el patrón de diseño *Factory Method*. Esta fábrica se encarga de instanciar personajes de tipo `Mago` o `Mortífago` de manera aleatoria, asignándoles automáticamente sus hechizos iniciales de acuerdo a esta característica.

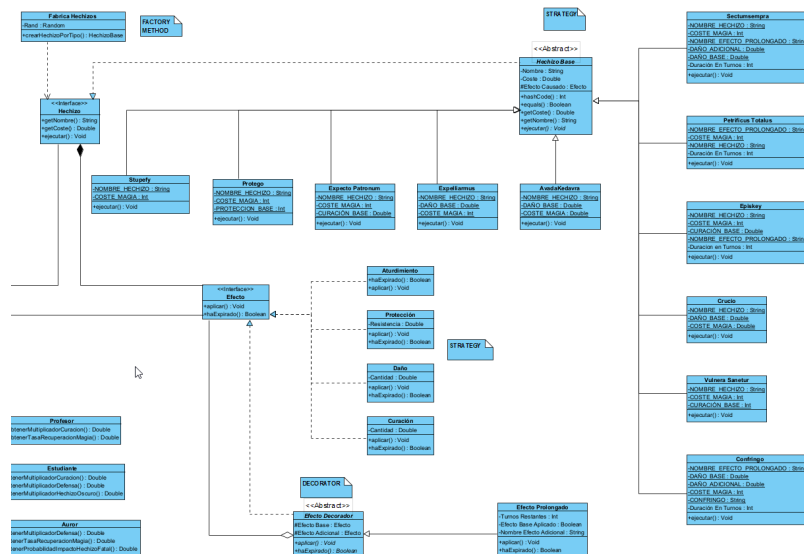
La administración de los equipos y la validación de las reglas del combate se modelaron a la clase Batallon, haciendo uso de tres colecciones.

- [illegible]

2.3. Diseño de Hechizos y Gestión de Recursos (Maná)

Al igual que con los personajes, la instanciación de las habilidades se centralizó mediante el patrón *Factory Method* en la clase Fábrica Hechizos, permitiendo que el sistema sea fácilmente extensible a futuro.

Finalmente, de forma adicional, se incorporó una mecánica de gestión de recursos (Maná). Los personajes cuentan con una reserva de magia que disminuye según el costo del hechizo lanzado y se regenera en turnos donde el maná es insuficiente.



Por último, la sección de Efecto surge gracias al bonus. El objetivo era desarrollar la funcionalidad en la que los hechizos dejaban al objetivo en uno o más estados durante varios turnos y la forma en que lo implementamos sigue el patrón de diseño *Decorator*. Este consiste en crear una clase encapsuladora en la que se almacenen todos los estados en los que se encuentra el personaje y a través de él es que podemos agregar, eliminar y hacer que convivan varios estados en simultáneo.

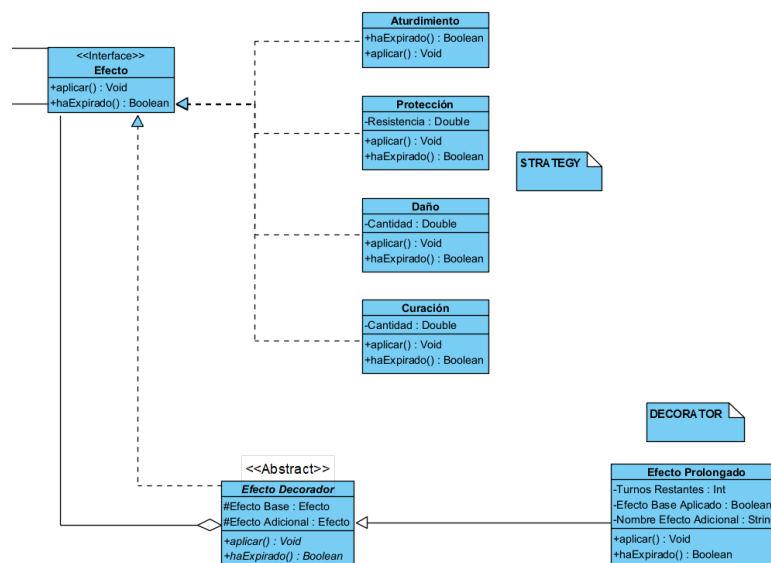
A nivel arquitectura, se creó la clase EfectoProlongado cuyo propósito es actuar como un envoltorio. Dicha clase requiere en la instanciación dos componentes: un efectoBase (el impacto instantáneo del hechizo) y un efectoAdicional (el daño o curación sostenida).

Mediante el uso de una bandera, la clase garantiza que el golpe inicial se aplique una única vez, mientras que delega la aplicación del estado continuo a los turnos posteriores hasta que el contador de duración llegue a cero.

Se destacan dos decisiones de diseño:

- **Delegación del cálculo matemático:** Para evitar el acoplamiento y la necesidad de que el efecto conozca al lanzador original, la resolución matemática de los multiplicadores (ataque, defensa, afinidad oscura/luminosa) se realiza íntegramente dentro del método ejecutar() de la clase del hechizo en cuestión. El efecto simplemente recibe el valor final calculado, independizando la mecánica del estado alterado respecto de su origen.
- **Manejo seguro de concurrencia:** En la clase Personaje, la gestión del ciclo de vida de los estados temporales se resolvió mediante el uso de la interfaz *Iterator*. Esto permitió recorrer la lista de efectos activos y remover aquellos que ya habían expirado.

En cuanto al comportamiento particular de cada efecto, dado que todos comparten los nombres de los métodos pero cada uno funciona internamente de forma distinta, se volvió a usar el patrón de diseño *Strategy*.



3. Conclusiones

El desarrollo de este simulador de batallas representó una instancia integradora que nos permitió comprender de mejor manera herramientas vistas en clase en la resolución de problemas complejos y la construcción de software escalable.

Durante el transcurso del proyecto, nos enfrentamos a diversos desafíos, siendo uno de los más significativos fue la correcta implementación de colecciones, particularmente el uso de la interfaz *Set* para evitar la duplicación de hechizos en un mismo turno. Este obstáculo exigió comprender a fondo el funcionamiento interno de Java y la necesidad de

sobreescribir los métodos equals() y hashCode() para comparar objetos por su estado lógico en lugar de su referencia en memoria.

La aplicación del polimorfismo eliminó por completo la necesidad de código condicional repetitivo, mientras que la delegación de responsabilidades a través del patrón Factory Method dejó el sistema preparado para su evolución. A esto se suma la flexibilidad del patrón Decorator para los estados temporales, lo cual garantiza que el sistema permite la adición de nuevos estados sin necesidad de modificar el código de los ya existentes. Si en un futuro se deseara incorporar una nueva facción, un nuevo personaje o un nuevo tipo de hechizo, el impacto sobre el código principal de la batalla sería leve.

4. Referencias

- Cátedra de Paradigmas de Programación. (s.f.). *Workspace* [Repositorio de código fuente]. GitHub. Recuperado el 30 de junio de 2026, de <https://github.com/paradigmas-de-programacion/workspace>
- Shvets, A. (s.f.). *Factory Method*. Refactoring.Guru. Recuperado el 30 de junio de 2026, de <https://refactoring.guru/es/design-patterns/factory-method>
- Shvets, A. (s.f.). *Decorator*. Refactoring.Guru. Recuperado el 30 de junio de 2026, de <https://refactoring.guru/es/design-patterns/decorator>
- Shvets, A. (s.f.). *Strategy*. Refactoring.Guru. Recuperado el 30 de junio de 2026, de <https://refactoring.guru/es/design-patterns/strategy>